

Bridging the Domain Language Divide in Automatic Multiplatform Single Page Application Generation with Knowledge Graph

Le Yen Chi Pham¹, Duc Minh Le², Van Cong Nguyen², Minh Khue Hoang², Dang Duc Anh Nguyen², and Quang Tung Ta²

¹ Swinburne University of Technology, Hawthorn, Victoria, Australia
103430040@student.swin.edu.au

² Department of Information Technology, FPT University, Swinburne Vietnam, Hanoi, Vietnam
{duc1m20, congnv24}@fe.edu.vn, {khuehmswh00983, anhnddsw01109, tungtqsw01093}@fpt.edu.vn

Abstract. Automating the software development process has long been a central topic in software engineering. The advent of modern AI methods and tools, especially the recent rapid advancement of generative AI has brought renewed research interests in the automation capability. A main limitation of the current AI-based approaches is that they are only able to generate code fragments and not the complete software module. Our recent works have proposed a novel software generation solution for a popular type of web frontend application, named single page application (SPA). Our works use annotation-based domain specific languages (aDSLs) to express the SPA design specification and code generators to automatically generate complete SPA software modules. However, our works currently lack the ability to express the SPA specifications in higher-level languages, necessary to leverage the AI methods and tools. In this paper, we aim to make a first novel step in bridging this gap by using knowledge graph to express the SPA specification, bringing it closer to the domain expert languages. We construct the knowledge graph model and implement it in a Java software framework, named JDA. After that, we develop an algorithm named SPKGen to generate aDSL-based SPA specification from the knowledge graph model. Subsequently, we evaluate the effectiveness of the knowledge graph model against real-world SPA case studies. We assess the knowledge graph model’s ability to accurately represent the configurations of SPA. Further, we evaluate the effectiveness of our method in generating the SPA components and configurations from the knowledge graph. The results of these evaluations demonstrate the viability of using knowledge graph for capturing and realizing SPA.

Keywords: Knowledge graph, Domain-Driven-Design, Generative software development, Single Page Application.

1 Introduction

Domain-Driven Design (DDD) provides principles and practices for building complex software systems from a core domain model [3]. Generative programming techniques,

such as code generation and metaprogramming, can help automate software development for these complex applications [2]. Additionally, Domain-Specific Languages (DSLs) allow for the creation of languages tailored to specific problem domains, which can be beneficial for automating the development of Single Page Applications (SPAs) [5]. Our previous work has proposed a novel method to automatically generate frontend applications for multiple SPA frameworks using annotation-based DSLs (aDSLs) and code generators implemented in the JDA framework [8,9,10]. However, a key limitation of our work is the lack of a formal mapping between the aDSL specifications and higher-level design models, such as those expressed in UML or knowledge graphs [12,13]. Although knowledge graph has been used in software development in general, to the best of our knowledge, there have been no work researching the application of knowledge graph to SPA. This paper aims to bridge this gap by using knowledge graph to formally express the aDSL-based SPA specifications. Specifically, we propose a knowledge-graph-based language, named Software Configuration Graph Language (SCGL), for representing both the metamodel and model elements of SPA specification. To ease SPA application in SCGL, we define an algorithm named SPKGen, which transforms a software configuration graph, expressed in SCGL, into a software configuration class model. We leverage our recent research [9,10] to automatically generate SPAs from this model. We evaluate the performance of SPKGen and the effectiveness of SCGL for expressing SPA design. The results demonstrate the overall effectiveness of our approach.

The rest of this paper is structured as follows. In Section 2, we provide an overview of the relevant concepts and prior research that serve as the foundation for this work. Section 3 defines SCGL as a knowledge-graph-based language for SPA specification. Section 4 presents the SPKGen algorithm. In Section 5, we assess the effectiveness of SCGL for expressing SPA specification and the performance of SPKGen algorithm. Finally, Section 6 concludes the paper and outlines potential future research directions.

2 Background and Related Works

Our work presented in this paper lies at the interactions of these topic domains: (1) multiplatform SPA generation and (2) application of knowledge graph in the development of single page application (SPA).

2.1 Expressing SPA design using SCCL*

In our previous work, we proposed a fundamental metamodel for SPAs which is expressed using an annotation-based DSL called SCCL* [9]. This metamodel was derived from the requirements of four popular front-end frameworks: Angular, React, React Native, and Vue.js (referred to as the "ARNV" frameworks). The SPA metamodel describes the foundational architecture of SPAs, including key elements such as modules, components (main form, object form, object browser), routing, and backend communication. However, building SPAs using this metamodel remains complex, as modern front-end development often involves multiple visual components provided by third-party libraries, such as Bootstrap. To address this, we adopted a systematic approach that integrates front-end design patterns, allowing a more structured representation of

visual components, whether custom-built or third-party. This approach streamlines development through code generation and automation.

This paper uses the CourseMan software example from our previous work, focusing on six key classes: SCCCourseMan, ModuleStudent, ModuleCourse, ModuleAddress, and ModuleEnrolment, which represent entities in a course management system. To simplify, we will illustrate SCCCourseMan and ModuleStudent, as the other modules follow similar implementations. This example serves as a benchmark for evaluating SCGL and SPKGen algorithm.

Listing 1.1. SCCCourseman (Source: [9])

```
1 @SystemDesc(
2   appName="Courseman",
3   dsDesc = @DSDesc(
4     type="postgresql",
5     dsUrl = "http://dataserver:5432/
6       courseman",
7     user = "admin",
8     password = "password",
9     connType=Client
10  ),
11  modules = {
12    ModuleMain.class,
13    ModuleAddress.class,
14    ModuleCourseModule.class,
15    ModuleEnrolment.class,
16    ModuleStudent.class
17  }
18 )
19 public class SCCCourseMan { }
```

Listing 1.2. Student Module (Source: [9])

```
1 @ModuleDesc( name="Module: Student",
2   model="Student", view=View.class,
3   containmentTree = @CTree {
4     root = "Student",
5     edges = {
```

```
6   @CEdge (p=Student.class, c=
7     Address.class),
8   @CEdge (p=Student.class, c=
9     Enrolment.class)
10  })
11  class ModuleStudent {
12    @AttributeDesc(label="Form: Student")
13    private String title;
14    @AttributeDesc(label="Id", type=
15      JTextField.class)
16    private int id;
17    @AttributeDesc(label="Name", type=
18      JTextField.class)
19    private String name;
20    @AttributeDesc(label="Gender",
21      type=JComboBox.class)
22    private Gender gender;
23    @AttributeDesc(label="Dob")
24    private Date dob;
25    @AttributeDesc(label="Address", type=
26      JDataPanel.class)
27    private Address address;
28    @AttributeDesc(label="Form: Enrolment")
29    private Collection <Enrolment> enrolls
30    ;
31  }
```

Listing 1 demonstrates the SCCL* configuration for SCCCourseMan and its relationship with the various modules. Similarly, Listing 2 shows the SCCL* configuration for the Student module within the CourseMan software. These SCCL* configurations are written using Java's syntax.

Figure 1 shows the SPA view of the Student module of the CourseMan application, which is generated from the SCCL* configuration using the React framework by the JDA framework introduced in our previous work [9].

2.2 JDA: Generating multiplatform SPAs from SCCL*

The SPA development process in this approach begins by creating an SPA model using the SCCL* language, supported by the JDA framework [9]. This model is then input into the SPAGen code generator, a JDA module, which produces the actual SPA code based on the specified front-end platform.

In our previous research [9,10], we have proposed a method for automatically generating frontend applications across multiple frameworks. This approach utilizes a set of annotation-based DSLs (aDSLs) to express the SPA model. We then developed code generators in object-oriented programming languages (OOPLs) to parse the aDSL specifications and generate the corresponding frontend software. The aDSLs allow for the specification of the target frontend framework, such as Vue.js, React, or Angular. The

The screenshot shows a web form titled "Form: Student". At the top, there are three buttons: "Main" (blue), "Browse" (blue), and "Delete" (red). To the right of these buttons is a search bar containing "S2021" and a magnifying glass icon. Below the search bar, the form fields are as follows:

- Id:** A text input field containing "S2021".
- Name:** A text input field containing "Le Minh Duc".
- Gender:** A dropdown menu with "Male" selected.
- Dob:** A date input field containing "10/12/2000".
- Address ID:** A dropdown menu with "1" selected.
- Address:** A text input field containing "1-Hanoi".

At the bottom right of the form, there is a red "Unlink" button and a dropdown arrow. At the very bottom, there is a green button labeled "Form: Enrollment".

Fig. 1. The Student module of CourseMan example in Reactjs (Source: [9]).

code generators incorporate well-defined UI design patterns to improve the usability of the generated software. We have implemented our aDSLs and code generators within a Java software framework named JDA [8,9].

However, a key limitation is the lack of a formal method to map the aDSL specifications to higher-level design models, such as those expressed in UML [12] or knowledge graphs [13]. Establishing a formal mapping between the aDSL specifications and these higher-level design models could provide significant benefits. It would allow us to leverage the capabilities of our code generators not only for frontend software development.

Knowledge graphs, in particular, offer a powerful way to represent and reason about the relationships and domain knowledge within software applications. Mapping aDSLs to a knowledge graph model would capture contextual information between components and business logic, improving code generation and enabling semantic queries. This formal mapping could also advance generative AI in software engineering. The next section discusses how knowledge graphs are applied in generating SPAs.

2.3 Knowledge Graph and Its Application in SPA Construction

A knowledge graph (KG) is a structured representation of knowledge, typically used to organize information in a machine-readable format. It is essentially a graph database that connects entities, concepts, and their relationships. In a knowledge graph, entities are represented as nodes, and the relationships between them are represented as edges [4]. Knowledge graphs have numerous applications, particularly in the field of Artificial Intelligence (AI). They facilitate tasks such as question answering, recommendation systems, and semantic search by organizing structured knowledge to enable efficient information retrieval and analysis, thereby enhancing various aspects of artificial intelligence [6]. By capturing and representing diverse types of knowledge and relationships, KGs can also be highly beneficial throughout the software development lifecycle [13], particularly in requirements engineering (modeling relationships between requirements and system components [7]), in software design and architecture (visualizing architecture and dependencies [1]), and in maintenance and evolution: tracking changes and bug reports linked to code.

Although KG has been quite extensively used in software development, to the best of our knowledge it has not been used to express the metamodel of aDSLs for frontend applications. Specifically, we used two keyword queries on Google Scholar to find the relevant articles. The first query aims to find specific articles on KG for Single Page Applications (SPA) development. The query is: "Knowledge Graph" AND (SPA OR "single page application") AND (generation OR construction OR development). We reviewed the first 200 results but none of these works actually discuss KG for SPA development or generation. These results only show works in either (i) KG construction for domain-specific knowledge-base (e.g. tourism, procurement value chain, online tutoring assistant, question answering, etc.) or (ii) KG engineering (including automatic KG generation, visualisation, exploration and editing, query answering over KG, etc.). The second query extends the first to cover also frontend: "Knowledge Graph" AND (SPA OR "single page application" OR "frontend" OR "front end") AND (generation OR construction OR development). We also reviewed the first 200 results. This second search returns a few additional articles concerning the general application of KG to software development (e.g. [11]). However, they do not discuss specifically frontend or SPA development.

This paper presents our first important step in bridging the gap in application of KG to SPA development. This is especially beneficial to the development of multiplatform SPAs, where there is a need to uniformly represent the cross-platform features. More specifically, we focus on aDSL-based software generation for multiplatform SPAs. We propose to formally express the aDSL specification in a knowledge graph format. The goal is to leverage the capabilities of knowledge graphs to express SCCL* for the purpose of generating SPAs. Resolving this issue will help lay a foundation for building a generative AI capability on top of our existing multiplatform code generators.

3 Representing SCCL* in SCGL

We use knowledge graph to express the SPA design specification embedded in SCCL*. We call the new language **Software Configuration Graph Language (SCGL)**. The benefit of knowledge graph is that it helps represent both the underlying language model (a.k.a the metamodel) and its models in the same graph. A model represents a software model that is instantiated from the metamodel for a specific problem domain. Figure 2 shows a diagram to visualize the design of SCGL using nodes and their relationships. Within the scope of this paper, we focus on the core elements of SCCL*. We use the popular Neo4j graph modeling language³ as the base knowledge graph language. To ease discussion, we will loosely use the term Neo4j to refer to this graph language.

To express the SCCL* model in Neo4j, we capture the concepts and relationships in SCCL* and map them to Neo4j node and relationship types. We create the necessary node labels and relationship types to match the entities and associations defined in the SCCL*'s metamodel. SCGL's metamodel expresses the design and constraints of SCCL*. In SCGL, a model defines the SCC model for a specific software.

Key relationships between entities, such as inheritance, composition, containment, and annotation, correspond to Neo4j relationships like "INSTANCE_OF," "HAS,"

³ <https://neo4j.com/use-cases/knowledge-graph/>

"HAS_CONTAINMENT_TREE" and "ATTACH_TO," preserving semantic connections and cardinality constraints. However, Neo4j does not natively support modeling with constraints, so we layered the graph design into three levels, each with specific rules to structure SCGL.

The central entities in the metamodel were directly translated into node labels in Neo4j, ensuring an accurate representation of concepts and their taxonomic structure. For example, the SCCCourseMan node is an SCG model for CourseMan. The target SPA configuration consists of a generation configuration (an instance of RFSGenDesc), a system configuration (an instance of SystemDesc), and module configurations, each represented by an instance of ModuleDescriptor with a data model.

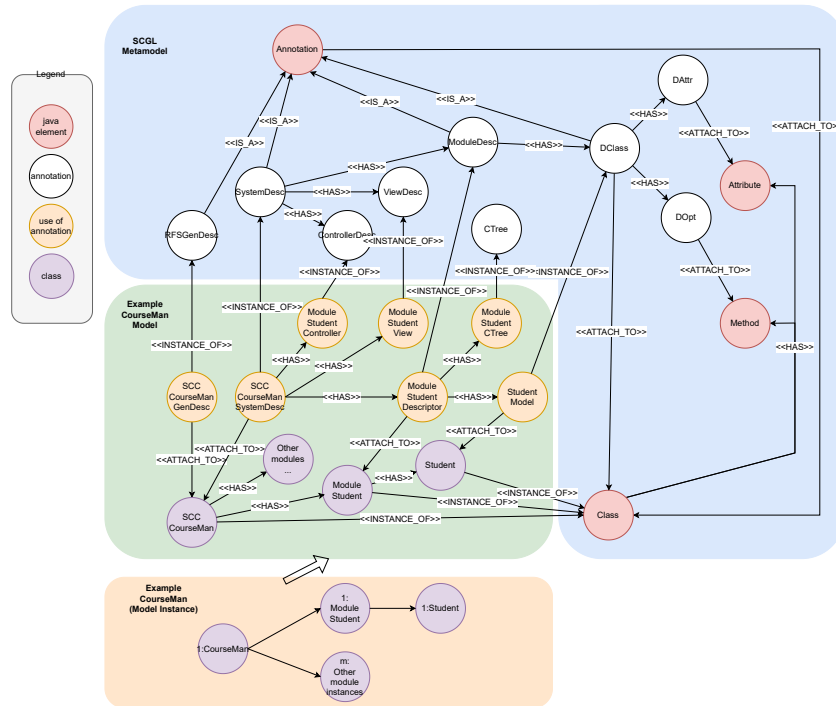


Fig. 2. SCGL structure and CourseMan example implementation.

3.1 Annotation, Class, Attribute, and Method node types

These node types represent various Java elements in SCGL. The other nodes may have the "IS_A" relationship with the element nodes to specify their type in Java. The Annotation nodes represent different types of Java annotations. They have an "IS_A" relationship with the "Annotation" node. Examples of annotation type nodes include "RFSGenDesc", "SystemDesc", and others, each named after the specific annotation they represent. The Class nodes represent Java classes. These class nodes have an "IS_A" relationship with the "Class" node.

3.2 Relationships

"ATTACH_TO" relationship with a class node, indicating which class the annotation is attached to. In addition, "INSTANCE_OF" relationship with an annotation type node, indicating the type of annotation.

Depending on the type of annotation instance, there may be additional relationships:

- "SystemDesc" annotation instances have one or more "HAS_MODULE" relationships with module class nodes. For example, a "SystemDesc" for "SCCCourseMan" may have "ModuleMain", "ModuleStudent", etc.
- "ModuleDescriptor" annotation instances have exactly one "HAS_MODEL" relationship with a domain class node, indicating the domain model for that module. They also have a "HAS_VIEW" relationship with a "ViewDesc" node and a "HAS_CONTROLLER" relationship with a "ControllerDesc" node.
- "CTree" (Containment Tree) annotation instances have one or more relationships "HAS_CONTAINMENT_EDGE" with other modules via domain class nodes. For example, a "CTree" for "ModuleEnrolment" may contain "Student", "Module".

3.3 Metamodel

Metamodel is the foundation of SCGL. It defines the structural elements, including concepts and their relationships in the metamodel of SCCL*. These include both the base elements of aDSL as well as those specific to SPA design. For SPA design, the captured elements include such as SetupDesc, ModuleDesc, ViewDesc, ControllerDesc, DClass, and DAttr, and their associations. These elements describe the different components and their interactions within the software application. The Metamodel provides the necessary framework to model the software configuration in a structured and consistent manner.

3.4 Model

The Model is an instance of the Metamodel, conforming to the design and constraints established in the Metamodel layer. This layer represents the specific configuration of the software, including the various software components and their relationships. Fig.2 shows an example model for the CourseMan SPA example. It includes nodes, such as SCCCourseManGenDesc, SCCCourseManSystemDesc, ModuleStudentDescriptor, and Student Model. These nodes and their relationships define the specific entities and interactions involved in the software application, such as course management, module management, and student-related functionality.

4 SPKGen: From SCGL to SCCL*

To ensure that the Neo4j knowledge graph schema aligns with SCCL*, we developed a method to query the graph and reconstruct the configurations as SCCL* in Java. This algorithm, called SPKGen (Knowledge-Graph-Based Single Page Application Generation), generates SPAs from a knowledge graph written in SCGL by transforming the graph into SCCL*. The transformed SCCL* is then used, along with the JDA framework, to generate the target SPA. In this paper, we focus on the essential elements of SCCL* supported by SCGL, with additional features planned for future work.

Algorithm 1 SPKGen Algorithm

```

1: function EXTRACT_KNOWLEDGEGRAPH(KG_Connection)
2:   classNodes ← queryClassNodes(KG_Connection)
3:   for all classNode in classNodes do
4:     className ← classNode.getProperty("name")
5:     classPackage ← classNode.getProperty("package")
6:     outputFilePath ← classPackage + className
7:     path ← classPackage + className
8:     if path ≠ null then
9:       copyFile(path, outputFilePath)
10:    else
11:      continue
12:    end if
13:    classModel ← new ClassModel()
14:    classModel.name ← className
15:    classModel.package ← classPackage
16:    classModel.annotations ← new List<string>
17:    annotationPairs ← queryAnnotations(knowledgeGraph, classNode)
18:    for all (annotationNode, annotationTypeNode) in annotationPairs do
19:      annotationTemplate ← getAnnotationTemplate(annotationTypeNode.name)
20:      annotationModel ← AnnotationModel.nodeToModel(annotationNode, anno-
        tionTypeNode.name)
21:      annotationBoundOutput ← bind(annotationTemplate, annotationModel)
22:      classModel.annotations.add(annotationBoundOutput)
23:      writeToFile(outputFilePath, bind(classTemplate, classModel))
24:    end for
25:  end for
26: end function

```

We use the Cypher query language⁴ of Neo4j to query the knowledge graph of SCGL to retrieve the design information. This allows us to programmatically extract the node labels, relationship types, and their associated properties that represent the schema using our algorithm. The detailed description and the steps of SPKGen are as follow:

4.1 Query all class nodes from the knowledge graph

The algorithm starts by querying the knowledge graph to retrieve all the class nodes. This is done using a Cypher query like `MATCH [n:CLASS]-[:IS_A]->[e:JAVA_ELEMENT name: "Class"] RETURN n`.

4.2 Check each class node for the "path" property

For each class node retrieved in the previous step, the algorithm checks if the node has a "path" property. If the "path" property exists, it means this class will not be generated, but instead, it will be programmatically copied over to the output directory from the Java file indicated by the "path" property. If the "path" property does not exist, the algorithm proceeds to the next step to generate this class.

4.3 Generate the classes

For each class node that needs to be generated (i.e., does not have the "path" property), the algorithm reads the class name from the node and puts it in a Freemarker template. It then queries the knowledge graph to retrieve the annotations and their corresponding annotation types associated with the class.

For each pair of annotation and annotation type, the algorithm determines the right Freemarker template to use based on the annotation type. If the annotation type is "RFS-GenDesc", the algorithm simply populates the corresponding template with information from the annotation node. If the annotation type is "SystemDesc", the algorithm queries for all classes with the relationship (this SystemDesc instance)-[:HAS_MODULE]->(n:CLASS). The resulting array of nodes will become the "modules" property of

⁴ <https://neo4j.com/product/cypher-graph-query-language/>

the annotation, and the other properties can be retrieved from the annotation node. If the annotation type is "ModuleDescriptor", the algorithm queries for the class with the relationship (this ModuleDescriptor instance)-[:HAS_MODEL]->(n:CLASS). If the knowledge graph is correct, there should be only one n, which is a domain class. The name of this domain class will become part of the ModuleDescriptor's ModelDesc, and the other information of ModuleDescriptor can be either static or retrieved from the annotation node. Similarly, If the annotation type is "CTree", the algorithm queries for all classes with the relationship (this CTree instance)-[:HAS_CONTAINMENT_EDGE]->(n:ANNOTATION). The resulting array of nodes will become the "edges" property of the annotation, and the other properties can be retrieved from the annotation node. The output of the annotation processing is a string result from binding the annotation template with the data. This string is appended to the \$annotations part of the class template. Finally, the complete class is written to the specified Java source file.

5 Evaluations

The primary focus of the evaluation in this work is to assess the effectiveness of the proposed approach in meeting the SCGL requirements outlined in Section 4. To that end, we have formulated the following questions to guide our evaluation:

EQ1: (SCGL) What is the extent to which SCGL correctly expresses the SPA design through SCCL*?

EQ2: (SPKGen) What is the effectiveness of the SPKGen algorithm in generating the SCC* for a given SPA?

EQ3: (SCGL for code generation) How effective is SCGL in generating SPA code?

We have implemented SCGL and SPKGen as part of JDA, and the code is available online in the GitHub repository: <https://github.com/jdomainapp/jdm-sgl>. Our implementation uses Java and add the following third party libraries to JDA:

- `org.neo4j.driver`: This Neo4j library provide a driver that helps connecting to the AuraDB and query the knowledge graph from it.
- `org.freemarker`: This library allow creating template for the generated code file.

5.1 The expressiveness of SCGL for SPA design

To assess the effectiveness of the SCGL knowledge graph in representing the SCCL* software configuration model, we developed a systematic approach using a mapping table. Table 1 establishes a clear correspondence between the key SCCL* concepts and their equivalent representations in the SCGL knowledge graph. The one-to-one correspondence between the SCGL and SCCL* concepts facilitates the translation and integration of software configuration data between the two languages.

Table 1 thus serves as a reference for developers and researchers working with SCGL and SCCL* to ensure that the SCGL knowledge graph can effectively capture and represent the software configuration information as defined in the SCCL* model.

5.2 SPKGen Testing

To evaluate the effectiveness of the proposed SPKGen algorithm, we adopted the Course-Man example from our previous work as a benchmark. In this evaluation, we first created

Table 1. SCGL and SCCL* OOP and Software Configuration Elements

No.	SCGL	SCCL*
OOP Elements		
1	"Annotation" node type, "ATTACHED_TO" relationship	Annotation
2	"Class" node type	Class
3	"Attribute" node type, "HAS" relationship	Attribute
4	"Method" node type, "HAS" relationship	Method
Software Configuration Elements		
5	"RFSGenDesc" node	RFSGenDesc annotation
6	"SystemDesc" node	SystemDesc annotation
7	"ModuleDesc" node	ModuleDescriptor annotation
8	"ViewDesc" node	ViewDesc annotation
9	"ControllerDesc" node	ControllerDesc annotation
10	"DClass" node	DClass annotation
11	"DAttr" node	DAttr annotation
12	"DOpt" node	DOpt annotation

a SCG model to represent the CourseMan application. We then used the SPKGen algorithm to help JDA to generate the corresponding CourseMan application implementation.

Listing 1.3. Generated SCCCourseman

```

1 @SystemDesc( appName="Courseman",
2 dsDesc = @DSDesc( type="postgresql",
3 dsUrl="http://dataserver:5432/domainds",
4 user = "postgres",
5 password = "postgres"),
6 modules = { ModuleMain.class,
7 ModuleCourseModule.class,
8 ModuleEnrolment.class,
9 ModuleStudent.class, ModuleAddress.class,
10 ModuleStudentClass.class})
11 public class SCCCourseMan { }
```

Listing 1.4. Generated Student Module

```

1 @ModuleDesc(name="ModuleStudent",
2 modelDesc = @ModelDesc(
3 model = Student.class),
4 viewDesc = @ViewDesc(
5 formTitle = "Form: Student",
6 domainClassLabel = "Student",
7 imageIcon = "Student.png"),
```

```

8 controllerDesc = @ControllerDesc(
9 controller = ControllerBasic.class),
10 type = ModuleType.DomainData)
11 public class ModuleStudent {
12 @AttributeDesc(label = "Student class")
13 private String title;
14 @AttributeDesc(label="Id", type =
15 JTextField.class)
16 private int id;
17 @AttributeDesc(label = "Name", type =
18 JTextField.class)
19 private String name;
20 @AttributeDesc(label = "Gender", type=
21 JComboField.class)
22 private Gender gender;
23 @AttributeDesc(label = "Dob")
24 private Date dob;
25 @AttributeDesc(label = "Address", type=
26 JDataPanel.class)
27 private Address address;
28 @AttributeDesc(label = "Enrolments")
29 private Collection <Enrolment> enrols; }
```

We compare the generated result with the original CourseMan in Section 2. The comparison showed that the generated application support all the basic elements listed earlier in Table 1.

5.3 SPKGen Performance

We further conduct a time complexity evaluation of the SPKGen algorithm to assess its computational efficiency and scalability. These performance characteristics are critical for the practical application of the SPKGen algorithm in real-world software development scenarios.

Theorem 1. (SPKGen Performance) *The time complexity of our SPKGen algorithm is quadratic to the number of class nodes in the input graph.* $\square$

Proof. Assuming the time complexity of writing to a file is $O(1)$, the overall time complexity of the SPKGen algorithm can be broken down as follows.

In the first step, the algorithm queries all class nodes in the Neo4j knowledge graph. Let the number of class nodes be denoted as n , and the time complexity for this step is $O(n)$. Next, the algorithm loops through the list of class nodes, which also incurs a time complexity of $O(n)$. For each class node, it performs several sub-steps. First, it reads basic class information such as the name and package from the node, which is $O(1)$. Then, it checks for the existence of a “path” property, again $O(1)$. If the property exists, the class is copied from the original file (with time complexity $O(1)$), otherwise, the algorithm proceeds to generate the class. Class generation involves creating a class model object, which is $O(1)$, querying the class annotations (with time complexity $O(m)$, where m represents the number of annotations), and for each annotation, creating and binding a model object to a template, also $O(m)$. Finally, the class model is bound to a template and written to the output file, with a time complexity of $O(1)$.

Assuming that k nodes have the “path” property, and $n - k$ nodes do not, the overall time complexity of the SPKGen algorithm is the sum of the time complexities of the individual steps, resulting in:

$$\begin{aligned} &O(n) + O(k \cdot (1 + 1 + 1) + (n - k) \cdot (1 + m + m + 1)) \\ &= O(n) + O(2nm + 2n + k - 2km) \end{aligned}$$

This simplifies to $O(nm)$, which can be approximated to $O(n^2)$, since in typical software, n is generally greater than m . $\square$

5.4 Effectiveness of Knowledge Graph

An experiment was conducted to evaluate the effectiveness of incorporating KG in the code generation process using ChatGPT. The objective was to test the hypothesis that **Generative AI (GenAI)** that is augmented with our SCGL would produce more optimized and accurate outputs compared to GenAI relying solely on natural language prompts.

Methodology: The experiment focused on generating frontend React code for modules including Student, Address, Course, Enrollment, and StudentClass. The generated code was tested and analyzed to identify key technical features and make comparisons. The results are available in a GitHub repository <https://github.com/jdomainapp/doc-scgl-soict2024>, organized into two folders: **CourseManScenario1** and **CourseManScenario2**, corresponding to the two different conditions below.

Scenario 1: Natural Language Prompts ChatGPT was provided with prompts written entirely in natural language, without additional contextual information.

Scenario 2: Knowledge Graph-Enhanced Prompts This scenario extended the experiment by incorporating structured knowledge from knowledge graphs, which captured both high-level and detailed relationships between the modules.

Results:

In Scenario 1, generating complete modules required 5 iterations. The initial prompt produced incomplete fields and unclear relationships between modules, requiring a follow-up to define these relationships in natural language. Even then, ChatGPT struggled with complex structures like the parent-child relationship between Course and its subclasses. Instead of distinct modules, ChatGPT generated a form with a Course Type field, showing the limitations of project enhancement.

Table 2. Comparison of Scenario 1 and 2 Results

Aspects	Scenario 1	Scenario 2
Total Prompts Used	5	3
Parent-child Relationships (binary)	0	1
Code Repetition (%)	15%	5%
API Integration (binary)	0	1
Modules Generated	6	8
Validation Coverage (%)	50%	90%
GPT Prompts and Results	Link to ChatGPT	Link to ChatGPT
GitHub Repository	Link to CourseManScenario1	Link to CourseManScenario2

In Scenario 2, leveraging a KG reduced the process to 3 iterations, plus one more prompt for nested forms. The code was more complete, accurately capturing complex relationships like the Course and its subclasses, reducing redundancy, and integrating API hooks for backend scalability. ChatGPT can also generate nested forms combining relevant modules like Student and Enrollment. Enhancements were easier, as the KG clarified module relationships, reducing the need for further prompts.

This experiment demonstrates that KG-enhanced GenAI provides a more powerful and efficient approach to code generation, offering better understanding, accuracy, and less manual intervention than standard GenAI. Further integrating KG into GenAI workflows could boost its effectiveness for practical applications in software development.

Discussion Combining the evaluation with the results from JDA and SCGL highlights the substantial benefits of utilizing KG for structuring the code generation process. While the code generated by JDA stands out for its superior user interface design compared to the GPT-generated code (see folders [CourseManJDA](#) and [CourseManScenario2](#) of our GitHub repository), both approaches exhibit the strengths in terms of performance. This includes aspects such as code optimization and the volume of modules generated, as illustrated in the previous sections for JDA and Table 2 for ChatGPT.

SCGL, in particular, demonstrates an advantage through its ability to represent complex relationships, which contributes to more optimized code and less repetition. This emphasizes the value of SCGL and the potential of KG in enhancing the automatic code generation process, especially when dealing with large-scale projects requiring intricate system architectures. Overall, the integration of SCGL with KG enhances efficiency and potential for further applications of KG in code generation.

6 Conclusion

In this work, we have presented a novel approach to automating the SPAs development process by using knowledge graph technology to express software design specifications. Our key contributions are the development of a knowledge-graph-based language (SCGL) that can capture the essential elements of SPA design, and the implementation of this language in a software framework (JDA) to generate SPAs. This framework allows domain experts to define SPA specifications in SCGL, and then automatically generates the complete SPA from these specifications. This work represents an important step towards bridging the gap between domain requirement language and design specifications in SPA development. Future extensions include enriching SCGL with SPA design patterns and investing the use of AI-powered software engineering tools with SCGL.

References

1. Ali Babar, M., Dingsøyr, T., Lago, P., Vliet, H.: Software Architecture Knowledge Management (Jan 2009). <https://doi.org/10.1007/978-3-642-02374-3>, journal Abbreviation: Software Architecture Knowledge Management Publication Title: Software Architecture Knowledge Management
2. Czarnecki, K.: Generative Programming: Methods, Techniques, and Applications Tutorial Abstract. In: Gacek, C. (ed.) Software Reuse: Methods, Techniques, and Tools. pp. 351–352. Springer, Berlin, Heidelberg (2002). https://doi.org/10.1007/3-540-46020-9_38
3. Evans, E.: Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional, Boston (2004)
4. Fensel, D., Şimşek, U., Angele, K., Huaman, E., Kärle, E., Panasiuk, O., Toma, I., Umbrich, J., Wahler, A.: Introduction: What Is a Knowledge Graph? In: Fensel, D., Şimşek, U., Angele, K., Huaman, E., Kärle, E., Panasiuk, O., Toma, I., Umbrich, J., Wahler, A. (eds.) Knowledge Graphs: Methodology, Tools and Selected Use Cases, pp. 1–10. Springer International Publishing, Cham (2020). https://doi.org/10.1007/978-3-030-37439-6_1, https://doi.org/10.1007/978-3-030-37439-6_1
5. Fowler, M., White, T.: Domain-Specific Languages. Addison-Wesley Professional (Sep 2010), <http://my.safaribooksonline.com/book/software-engineering-and-development/ide/9780132107549>
6. Ji, S., Pan, S., Cambria, E., Marttinen, P., Yu, P.S.: A Survey on Knowledge Graphs: Representation, Acquisition, and Applications. IEEE Transactions on Neural Networks and Learning Systems **33**(2), 494–514 (Feb 2022). <https://doi.org/10.1109/TNNLS.2021.3070843>, <https://ieeexplore.ieee.org/document/9416312>, conference Name: IEEE Transactions on Neural Networks and Learning Systems
7. Kasser, J.: A framework for requirements engineering in a digital integrated environment (freddie) (01 2000)
8. Le, D.M., Dang, D.H., Vu, H.T.: jDomainApp: A Module-Based Domain-Driven Software Framework. In: Proc. 10th Int. Symp. on Information and Communication Technology (SOICT). pp. 399–406. ACM, New York, USA (2019). <https://doi.org/https://doi.org/10.1145/3368926.3369657>
9. Le, D.M., Nguyen, A.P., Tran, L.Q., Le, H.T., Tong, H.V.A.: Generating Multi-platform Single Page Applications: A Hierarchical Domain-Driven Design Approach. In: Proc. Int. Conf. 11th SOICT 2022. pp. 344–351. SoICT '22, ACM, Hanoi, Vietnam (2022). <https://doi.org/10.1145/3568562.3568566>, <https://dl.acm.org/doi/10.1145/3568562.3568566>
10. Le, D.M., Nguyen, C.V., Tran, L.Q., Hoang, K.M.: Modularly Generating Multi-platform Single Page Applications with FrontEnd Patterns. In: Intelligent Systems and Networks. pp. 552–562. LNCS, Springer Nature, Singapore (2024). https://doi.org/10.1007/978-981-97-5504-2_65
11. Lin, Z.Q., Xie, B., Zou, Y.Z., Zhao, J.F., Li, X.D., Wei, J., Sun, H.L., Yin, G.: Intelligent Development Environment and Software Knowledge Graph. J. Comput. Sci. Technol. **32**(2), 242–249 (Mar 2017). <https://doi.org/10.1007/s11390-017-1718-y>, <https://doi.org/10.1007/s11390-017-1718-y>
12. Object Management Group (OMG): Unified Modeling Language version 2.5.1 (2017), <https://www.omg.org/spec/UML/2.5.1/PDF>, accessed: 2024-08-22
13. Wang, L., Sun, C., Zhang, C., Nie, W., Huang, K.: Application of knowledge graph in software engineering field: A systematic literature review. Information and Software Technology **164**, 107327 (Dec 2023). <https://doi.org/10.1016/j.infsof.2023.107327>, <https://www.sciencedirect.com/science/article/pii/S0950584923001829>